

Linguagem loo1

- LOO
- Exemplo de programa
- Ambiente de execução
- Comando New

LOO

- Linguagem Orientada a Objetos (loo1) estende a Imperativa 1 (li1) com declarações de classes, criação dinâmica de objetos, e chamada de métodos.
- Um programa é um comando precedido por declarações de classes.

LOO

- Atributos são similares às variáveis das Linguagens Imperativas 1 e 2.
- Métodos são similares aos procedimentos da Linguagem Imperativa 2;
- Métodos não são valores e podem ser recursivos e parametrizados,
- Mas só podem ocorrer dentro de declarações de classes.

LOO

```
Programa ::= "{" DecClasse ";" Comando "}"
Comando ::= "skip" | Atribuicao | IO | Comando ";" Comando
           | IfThenElse | While | ComandoDeclaracao
           | New | ChamadaMetodo
Atribuicao ::= LeftExpression ":" Expressao
LeftExpression ::= Id | AcessoAtributo
AcessoAtributo ::= LeftExpression.Id | this.Id
Expressao ::= Valor | ExpUnaria | ExpBinaria | LeftExpression | this
Valor ::= ValorConcreto
ValorConcreto ::= ValorInteiro | ValorBooleano | ValorString | ValorNull
ComDeclaracao ::= "{" DecVariavel ";" Comando "}"
DecVariavel ::= Tipo Id "=" Expressao | DecVariavel "," DecVariavel
                | Tipo Id ":" "new" Id
New ::= LeftExpression ":" "new" Id
ChamadaMetodo ::= Expressao "." Id "(" ListaExpressao ")" | Expressao "." Id "(" ")"
DecClasse ::= "classe" Id "{" DecVariavel ";" DecProcedimento "}"
                | DecClasse "," DecClasse
Tipo ::= TipoClasse | TipoPrimitivo
TipoClasse ::= Id
```

Figura 28. BNF para a Linguagem Orientada a Objetos

LOO

- A principal construção nova é o elemento que define declaração de classes.
- Com relação aos comandos da linguagem, a atribuição a atributos da classe é baseada em acesso qualificado à variável
 - (como em `conta.saldo`, assumindo que `conta` é uma instância da classe que tem `saldo` como atributo);
- Acesso à variável pode ocorrer também em expressões.

LOO

- Como em Java, **this** representa o objeto corrente
- Portanto, é também uma expressão válida na linguagem.
- O valor de um identificador de classe antes da criação de um objeto é representado por `ValorNull`.

LOO

- Em um comando declaração, a declaração de variáveis agora envolve também um tipo explícito.
- No caso de declaração de identificadores de instâncias de classes, objetos são criados a partir da construção **new** seguido do nome da classe.

LOO

- A criação de objetos não é uma expressão na linguagem:
- a construção é limitada à inicialização de uma declaração
- ou ao comando (New) que define uma forma particular de atribuição para a criação de objetos.

LOO

- A chamada de métodos segue a mesma sintaxe da chamada de procedimentos, já explicada anteriormente.
- Como de costume, a declaração de uma classe envolve o nome da classe, a declaração dos atributos e dos métodos;
- Os métodos seguem a sintaxe da declaração de procedimentos na Linguagem Imperativa 2 (li2).

Exemplo de programa

- Um exemplo de programa nesta linguagem é apresentado na figura a seguir.
- O programa declara uma classe que representa uma lista de valores (classe LValor)
- com atributos que representam o valor a ser armazenado e uma lista (a declaração da classe é recursiva).

Exemplo de programa

```
{ classe LValor {  
    int valor = -100,  
    LValor prox = null;  
    proc insere(int v) {  
        if (this.valor == -100) then { this.valor := v; this.prox := new LValor }  
        else {this.prox.insere(v)}  
    },  
    proc print() {  
        write(this.valor);  
        If (not(this.prox == null)) then {this.prox.print()} else {skip}  
    }  
};  
{ LValor lv := new LValor; lv.insere(3); lv.insere(4); lv.print() }  
}
```

Figura 29. Exemplo de programa na Linguagem Orientada a Objetos

Exemplo de programa

- Dois métodos são definidos:
- um método recursivo que inclui um elemento na lista
- e um método que imprime os elementos presentes na lista.
- O comando principal cria uma instância da classe e invoca um método para inserir elementos na lista e depois imprimi-los.

Ambiente de execução

- O ambiente de execução de um programa orientado a objetos é bem mais complexo do que o de um programa imperativo.
- Além dos métodos já considerados na Linguagem Imperativa 2, a interface do ambiente inclui métodos para:

Ambiente de execução

- mapear nomes de classes em definições de classes;
- retornar a definição de uma classe dada seu identificador;
- mapear identificadores, incluindo this, em valores, incluindo referências (este método é herdado e reutilizado da interface Ambiente);

Ambiente de execução

- mapear referências em objetos;
- retornar um objeto associado a uma referência;
- e retornar a referência para a próxima célula a ser alocada.

Ambiente de execução

```
public interface AmbienteExecucaoOO1 extends Ambiente<Valor>{  
    public void mapDefClasse(Id idArg, DefClasse defClasse) ;  
    public DefClasse getDefClasse(Id idArg);  
    public void mapObjeto(ValorRef valorRef, Objeto objeto);  
    public Objeto getObjeto(ValorRef valorRef);  
    public ValorRef getProxRef();  
}
```

Figura 30. Código da interface AmbienteExecucaoOO1

Comando New

- Parte da implementação do comando New é apresentada na Figura 31.

```
public class New implements Comando{
    private LeftExpression av;
    private Id classe;
    public AmbienteExecucaoOO1 executar(AmbienteExecucaoOO1 ambiente) {
        DefClasse defClasse = ambiente.getDefClasse(classe);
        DecVariavel decVariavel = defClasse.getDecVariavel();
        AmbienteExecucaoOO1 aux =
            decVariavel.elabora(new ContextoExecucaoOO1(ambiente));
        Objeto objeto = new Objeto(classe, aux);
        ValorRef vr = ambiente.getProxRef();
        ambiente.mapObjeto(vr, objeto);
        ambiente = new Atribuicao(av,vr).executar(ambiente);
        return ambiente;
    }
}
```

Figura 31. Código da classe New

Comando New

- Os atributos são semelhantes ao da classe que implementa uma atribuição,
- só que o lado esquerdo pode ser um acesso à variável (explicado anteriormente) e o direito contém o identificador da classe do objeto a ser criado.

Comando New

- O método executar recupera a definição da classe associada ao identificador, no ambiente.
- A partir desta definição, os atributos da classe, com os respectivos tipos, são recuperados.

Comando New

- Um mapeamento destes atributos em valores pré-definidos é introduzido em um ambiente auxiliar, a partir do qual o objeto é criado.
- Uma nova referência para o objeto é obtida e o objeto é mapeado no ambiente, associado a esta referência.

Comando New

- Por sua vez, a referência é associada ao acesso à variável que forma o lado esquerdo do comando New.
- Note que este mapeamento é feito como em uma atribuição.
- Finalmente, o ambiente é retornado.

Atividade 1

- Usando a loo1, elabore uma classe em Java chamada de **Exemplo1** para executar o seguinte comando precedido por declaração de classe:

Exemplo1

```
{
  classe Contador {
    int valor = 1;
    proc print() {
      write(this.valor)
    },
    proc inc() {
      this.valor := this.valor+1
    }
  };
}
```

```
{
  Contador c:= new Contador;
  c.inc();
  c.print();
}
}
```

Atividade 2

- Usando a loo1, elabore uma classe em Java chamada de **Exemplo2** para executar o seguinte comando precedido por declaração de classe:

Exemplo2

```
{
  classe Contador {
    int valor = 1;
    proc print() {
      write(this.valor)
    },
    proc inc() {
      this.valor := this.valor+1
    }
  };
};
```

```
{
  Contador c := new Contador,
  Contador c2:= new Contador;
  c.inc();
  c2.inc();
  c2.inc();
  c.print();
  c2.print()
}

}
```

Atividade 3

- Usando a loo1, elabore uma classe em Java chamada de **Exemplo3** para executar o seguinte comando precedido por declaração de classe:

Exemplo3

```
{ classe LValor {  
    int valor = -100,  
    LValor prox = null;  
  
    proc insere(int v) {  
        if ((this).valor == -100) then {  
            this.valor := v;  
            this.prox := new LValor  
        } else { (this).prox.insere(v) }  
    },  
}
```

Exemplo3 - cont.

```
proc print() {  
    write(this.valor);  
    if (not(this.prox == null)) then  
        { (this).prox.print() }  
    else {skip}  
}  
};  
  
{    LValor lv := new LValor;  
    lv.insere(3); lv.insere(4);  
    lv.print()  
}  
}
```

Atividade 4

- Usando a loo1, elabore uma classe em Java chamada de **Exemplo4** para executar o seguinte comando precedido por declaração de classe:

Exemplo4

```
{ classe LValor {  
    int valor = -100,  
    LValor prox = null;  
  
    proc insere(int v) {  
        if ((this).valor == -100) then {  
            this.valor := v;  
            this.prox := new LValor  
        } else { (this).prox.insere(v) }  
    },  
}
```

Exemplo4 - cont.

```
proc remove(int v) {  
    {  
        LValor aux = this;  
        while(not((aux.prox == null) or  
((aux).prox).valor == v))) do {  
            aux := aux.prox  
        };  
        if ( not( aux.prox == null) ) then {  
            aux.prox := ((aux).prox).prox  
        }  
        else { skip}  
    }  
  
},
```

Exemplo4 - cont.

```
proc print() {  
    write(this.valor);  
    if (not(this.prox == null)) then  
        { (this).prox.print() }  
    else {skip}  
}  
};  
  
{ LValor lv := new LValor;  
  lv.insere(2);lv.insere(3);  
  lv.insere(4);lv.print();  
  lv.remove(3);lv.print()  
}  
}
```


Subprojeto loo1

- Converter toda a linguagem loo1 que está escrita em Java para Python (inclusive com as classes de exemplos).
- Esse subprojeto completo deve ser entregue no final do curso.

Leitura recomendável

- Conceitos e Paradigmas de Programação via Projeto de Interpretadores
- Augusto Sampaio e Antônio Maranhão
- Material em preparação para o JAI-2008 (Jornadas de atualização em Informática – Sociedade Brasileira de Computação)